SLiM Workshop Series #6: Recombination

Copyright © 2020–2025 Benjamin C. Haller. All rights reserved.

1. Start a new model in SLiMgui

Run SLiMgui, and close any existing model windows. Press command-N to make a new model window. We will work with this model as a starting point.

This initial model defines a single uniform recombination rate along the whole chromosome:

```
initializeRecombinationRate(1e-8);
```

This tells SLiM that the probability of recombination between any given base position and the next base position is 10^{-8}, in each new haplosome being generated by meiosis, in each tick.

2. Set up a simple recombination map

We want to extend this model to define a simple recombination map that has a high rate at one end and a low rate at the other end. Click inside the parentheses of `initializeRecombinationRate()` and look at the function prototype shown at the bottom of SLiMgui's window:

```
(void)initializeRecombinationRate(numeric rates, [Ni ends = NULL],
   [string$ sex = "*"])
```

Thus far we have just been passing a singleton `float` for the rate, and using the default values for the other parameters. We will not use the `sex` parameter here – it is for setting up separate recombination maps for males versus females – but we do now need to use `ends`. This parameter is used to designate the *last* base position for each recombination rate specified in `rates`. It is easier to show this than to explain it, so replace the previous call with:

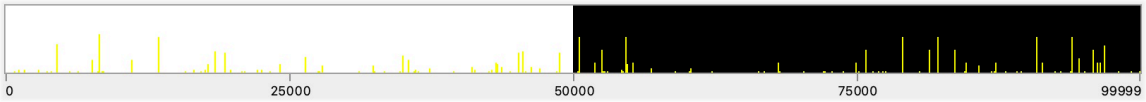
```
initializeRecombinationRate(c(1e-5, 0), c(49999, 99999));
```

Also change the tick for the final output event to `200000`, so that we can observe the model for a longer time (you don't have to run all the way to the end each time, of course). Recycle and run, and observe the model closely (perhaps using the play speed slider). The difference is subtle, but you may be able to see that in the left half of the chromosome mutations are drifting independently – their bars rise and fall individually – whereas in the right half of the chromosome mutations exhibit linkage into haplotypes that rise and fall in synchrony, as if their bars were connected.

Note how these values are interpreted. The rate `1e-5` is used as the probability of a crossover between each pair of bases from the pair (`0`, `1`) to the pair (`49998`, `49999`); the last use of the rate `1e-5` is to the *left* of base `49999`. After that, the rate `0` is used from pair (`49999`, `50000`) to the final pair, (`99998`, `99999`).

3. Visualize the recombination map in SLiMgui

It would be nice to be able to see the recombination map represented graphically, and SLiM can do this for you. Click the "action button" with the gear icon to the right of the chromosome view and select the Display Rate Maps option so it is checked, and you should see something like this:



Colors used by SLiMgui for this display range from white (very high recombination rates) through dark blue to black (very low recombination rates). The scale is somewhat arbitrary and the display is

more qualitative than quantitative, but it can at least provide a sense of whether the map is defined as intended.

EXERCISE: Change the recombination rate map in this model to define three regions instead of two. Use an intermediate rate of `1e-7` for the new region. Recycle and run to confirm visually that you have defined it as you intended.

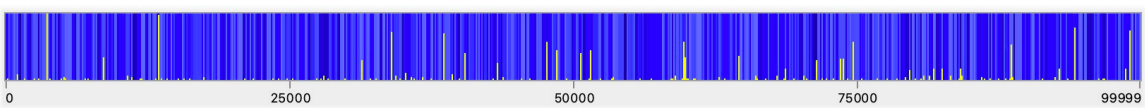
4. Define a random recombination rate map

Suppose we want to model a particular species – one we study in a real-world system, say. We have a sense of the general range of recombination rates that our species has along the chromosome, but a detailed recombination rate map has not yet been constructed for our species of interest. We'd like to know whether the variable recombination rate along the chromosome is likely to influence the results our model generates, by comparing a model with a random map to a model with a uniform recombination rate. We can set up a random recombination rate map very easily; just replace the previous call to `initializeRecombinationRate()` with this code:

```
rates = runif(1000, 1e-9, 1e-7);
ends = c(sort(sample(0:99998, 999)), 99999);
initializeRecombinationRate(rates, ends);
```

We will model a chromosome with 1000 recombination regions of different rates, with random lengths tiled across the chromosome. The `runif()` call generates `1000` random draws from a uniform distribution within the interval [`1e-9`, `1e-7`]. Then we generate 999 randomly chosen base positions in the range `0:99998` using `sample()`; since that samples without replacement by default, the base positions will all be different. We use `sort()` to sort them into ascending order, and append the final base position for the chromosome, `99999`, to the end of the vector. The end result is a vector 1000 elements long that provides the end positions for each recombination rate region, ending at the end of the chromosome.

Recycle and run this model with recombination rate map display enabled to see something like this:



The hotspots and coldspots defined by the random recombination rate map can be seen clearly. Of course this example is a bit artificial; you would likely want to use a more realistic distribution for both the rates and the endpoints. Nevertheless, it illustrates the concept. Later in this exercise we will see how to read and use an empirically measured recombination rate map from a file on disk.

EXERCISE: Change the maximum rate from `1e-7` to `1e-6`, and recycle and run. Note that the map is mostly white to light blue; most of the rates drawn will be at the top end of the range, because the interval [`1e-7`, `1e-6`] is almost ten times as wide as the interval [`1e-9`, `1e-7`].

EXERCISE: Given that observation, it might be nice to draw rates on a logarithmic scale instead, so that the low end of the range is well-represented. To do this, draw exponents using `runif()` with a min of `-9` and max of `-6`, and then raise `10` to the power of each exponent; remember that `^` is the exponentiation operator in Eidos. Recycle and run, and observe the difference.

5. Read a recombination rate map from a file

As a final example of working with recombination rate maps, we'll look at loading a rate map from a file. Close the current SLiMgui window and open recipe 8.2.2 from the **File** menu, under **Open Recipe**. You will need to download the input file it uses, which is a *Drosophila* recombination rate map published in Comeron et al. (2012). There is a URL in the recipe's script for the Recipes zip

archive (also linked to from the SLiM home page); copy/paste that URL into a web browser to download the archive, then double-click the archive to unzip it, locate the file inside the unzipped folder, and copy it to the Desktop where the model expects it (or add a path in the script that points to the correct location of the file, like `"/my/path/to/file.txt"`). If you open the file in a text editor, you can see that the lines in the file look like this:

```
1        0
100001   0
200001   0
300001   0.234550139
400001   0.234550139
500001   1.993676178
...
```

The positions specified in the file are starts, not ends, and they are one-based (the first base position on the chromosome is position `1`), whereas SLiM uses zero-based positions. The positions given in the first column therefore to be adjusted by removing the first value (the start position of `1`, which SLiM does not need) and adding the position of the end of the chromosome (`23011543`, which can be specified as `L-1` here since `L` is defined as the length of the chromosome with `defineConstant()` in this model). It is also necessary to subtract `2` from each of these positions – one to switch from one-based to zero-based positions, and another one to switch from starts to ends (since the position at the end of one recombination rate region is one less than the position at the start of the next region). Finally, the rates in the file are in units of cM/Mbp and must be multiplied by `1e-8` to convert them to SLiM's units of recombination rate per base position.

The core code in this recipe that reads and interprets the recombination rate map file is a single line that performs all of these conversions for us:

```
initializeRecombinationRateFromFile("Comeron_100kb_chr2L.txt", L-1);
```

This function, new in SLiM 5, makes it very easy to load recombination rate maps that are in this type of format; it is smart enough to do all of the necessary adjustments described above for us. A scaling factor of `1e-8` is the default, to convert from cM/Mbp, but you can supply a different scaling factor if needed.).

Recycle and run the model, and turn on display of the recombination rate map with the action button (choose "Display Rate Maps"), as before; if all is well, you should see the *Drosophila* 2L recombination rate map! Incidentally, separate recombination rate maps can be set for males and females with the `sex` parameter to `initializeRecombinationRateFromFile()`, so setting up sex-specific rates (as exist in *Drosophila*) is easy.

6. Read a recombination rate map from a file (a little history)

Prior to SLiM 5, this worksheet used to demonstrate how one might read in a file of this type using lower-level functions in Eidos. That is still useful to see – we will not see many examples of how to read data in from a file – so we'll walk through it quickly in the space remaining. **You don't need to enter this code; you can just read through this part. It is provided in the solutions as 006_step06_ex1.slim.** Here's the previous version of the code in this worksheet:

```
map = readCSV("Comeron_100kb_chr2L.txt", c("ends", "rates"), sep="\t");
ends = map.getValue("ends");
ends = c(ends[1:(size(ends)-1)] - 2, 23011543);
rates = map.getValue("rates") * 1e-8;
initializeRecombinationRate(rates, ends);
```

The `readCSV()` call reads in a CSV file, which stands for "comma-separated value", but we tell it to look for tabs (`sep="\t"`) as delimiters rather than commas. It returns an object called a `DataFrame`;

you can find documentation on the `DataFrame` class in the Eidos manual. `DataFrame` is used to store data that is organized into columns of values (just as in R). It is a subclass of another Eidos class, `Dictionary`, that lets you keep values organized by *keys* that can be used to look up the values. Here, we tell `readCSV()` the names of the columns in the file (since this file doesn't have a header line), which are "ends" and "rates", and the `DataFrame` it produces will use those column names as keys. We can then get the values for the two columns with `getValue("ends")` and `getValue("rates")`; the `getValue()` method is actually a method of `Dictionary`, inherited by `DataFrame`, so you can read about it in the documentation for `Dictionary`. Most SLiM classes inherit from `Dictionary`, and thus can remember values with `setValue()` and look them up with `getValue()`, as we will see later.

EXERCISE: The `initializeRecombinationRateFromFile()` function is actually written in Eidos, not in C++ the way most of SLiM is. You can therefore view the source code for it, if you're interested! If you type `functionSource("initializeRecombinationRateFromFile")` into the Eidos console, you can see what the code looks like. Spoiler alert: it is essentially the same as the `readCSV()` version of the code shown above, with a little extra logic for error-checking and such. It's quite easy to write your own user-defined functions to perform useful tasks like this in a reusable, shareable way; we'll look at this more later in the workshop.

7. Read a recombination rate map from a file (a little more history)

In an even older version of this worksheet, instead of using `readCSV()` and `Dictionary`, we read the file in using `readFile()` to get a string vector (one `string` element per line in the file) and processed it line by line to construct the ends and rates vectors passed to `initializeRecombinationRate()`. Again, you don't need to enter this code (it is 006_step07_ex1.slim), but it is instructive to look at it, to see how to read raw data from a file without even using `readCSV()`:

```
lines = readFile("~/Desktop/Comeron_100kb_chr2L.txt");
rates = NULL;
ends = NULL;

for (line in lines) {
   components = strsplit(line, "\t");
   ends = c(ends, asInteger(components[0]));
   rates = c(rates, asFloat(components[1]));
}
```

The first six lines of the file, shown earlier, would be the first six `string` elements in the vector `lines`. Let's consider what the loop does for the last of those six lines. The value of `line`, for that iteration of the `for` loop, would be `"500001\t1.993676178"`, where `\t` represents a tab character. The `strsplit()` function splits that string by tab characters (since `"\t"` is passed to it as the separator), producing a vector with two string elements, `"500001"` and `"1.993676178"`. The first of those strings, `components[0]`, is converted to an `integer` and appended onto the end of the variable `ends` with `c()`; the second, `components[1]`, is converted to a `float` and appended to the end of `rates` with `c()`.

This final version of the code demonstrates a general strategy for reading text files in Eidos, when even `readCSV()` can't be used: read in the lines of the file with `readFile()`, loop through them with a `for` loop, use `strsplit()` to split tab-separated or comma-separated values into separate strings, and handle those values however you wish. The new SLiM 5 version of recipe 8.2.2 using `initializeRecombinationRateFromFile()` is considerably simpler, but when a built-in function like that is not provided you sometimes need to process input files yourself, so it's good to know how to do such things!

6. BEEP! With extra time, read manual section 1.5.6, about the "DSB" recombination model. If you're interested in gene conversion, you can continue on to section 8.2.4.